AMERICAN LATIN CLASS ATTENDANCE SYSTEM

Architecture

Table of Contents

Architecture Documentation

Selected stack and justification

Node.js with Express is used for a clear academic backend that supports REST, middleware, validation, testing and modular OOP service classes. React + Vite is selected as the frontend framework for the next UI phase. PostgreSQL is the production database and Prisma ORM is the required data-access layer because it supports an explicit schema, generated client and AWS RDS deployment.

Backend architecture

Controllers only parse HTTP concerns and delegate to services. Services implement business rules. Repositories isolate persistence. Validators centralize request validation. Middleware handles sessions, authorization, cache control, CORS and errors.

Runtime layers:

- Controllers: `AuthController`, `CrudController`, `AttendanceController`, `ReportsController`, `AcademicController`.
- Services: `AuthService`, `AccessPolicy`, `AttendanceService`, `RulesService`, `AcademicService`, `AuditService`.
- Repositories: in-memory repositories for tests/local demos and Prisma repositories for production when `DB_DRIVER=prisma`.
- Middleware: validation, session resolver, RBAC, private page guard, no-store cache and error handling.

RBAC decides whether a role can enter a workflow. `AccessPolicy` then checks the specific resource scope: BranchDirector users are limited by `user_branch_access`, Teacher users are limited to their linked teacher profile/classes, and Student users are limited to their own academic records.

Python analytics microservice

An additional Python FastAPI API is included under `06Code/python-analytics-api` for analytical academic endpoints. It is not responsible for creating sessions or writing transactional attendance records. It reads from the same PostgreSQL database and validates the same JWT session token issued by the Node Auth API.

Runtime responsibility:

- Base path: `/api/analytics/v1`.
- Health endpoint: public service check.
- Protected endpoints: student attendance risk, scholarship readiness, branch performance and teacher workload.
- Session validation: JWT signature check plus server-side token hash lookup in the shared `sessions` table.
- Resource authorization: analytics responses are filtered by the same student, teacher and branch scope rules used by the Node API.

URI conventions

Node private APIs use `/api/v1`. The Python analytics API uses `/api/analytics/v1`. Both use plural nouns where applicable, JSON request/response bodies and consistent envelopes: `{ success, message, data }` or `{ success:false, message, data:null }`.

OAuth/session strategy

The React login page reads public OAuth configuration through `GET /api/v1/auth/config`, loads Google Identity Services and sends the Google ID token to `POST /api/v1/auth/google`. The backend verifies Google ID tokens with Google in production, links `google_sub` to an internal user and issues an application session through HttpOnly Secure SameSite cookies. For academic

Postman verification, `POST /api/v1/auth/login` can be enabled with configured email/password credentials and returns the same JWT session contract. Only a session token hash is stored server-side. Logout revokes the server-side session. Private pages use `Cache-Control: no-store` and a session guard that redirects anonymous users to `/login.html?session=expired`.

Database model

PostgreSQL schema is normalized around users/roles/permissions, explicit user branch access, branches, students, teachers, dance styles, class groups, class sessions, student attendance, teacher attendance, absence justifications, scholarship evaluations, level promotion evaluations, enrollment requests, sessions and audit logs.

AWS deployment

- Frontend EC2: Nginx on ports 80/443 serving static landing/app assets.
- Core Business API EC2: port 3000 for branches, students, teachers, classes and attendance.
- Auth & Session API EC2: port 3001 for OAuth, sessions, roles and permissions.
- Reports & Rules API EC2: port 3002 for scholarship, promotion, hours and payments.
- Python Analytics API EC2: port 8000 for attendance risk, scholarship readiness, branch performance and workload summaries.
- PostgreSQL: Amazon RDS private subnet on port 5432. Security groups should allow public 443 only to ALB/Nginx, API traffic only from frontend/ALB, and RDS only from API security groups. HTTPS with ACM certificates and an optional ALB is recommended.

Environment variables

See `06Code/.env.example` for required runtime variables.

Clean Code and SOLID decisions

Business rules live in services, HTTP details live in controllers, cross-cutting concerns live in middleware, and constants avoid magic strings. Classes depend on abstractions supplied through constructors.